

Deep Learning

Fundamentals of Machine Learning



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand underfitting and overfitting.
2. Understand regularization techniques.



Roadmap of this lecture:

1. Under-fitting and over-fitting.

2. Regularizations: L1, L2, dropout.



Underfitting and overfitting

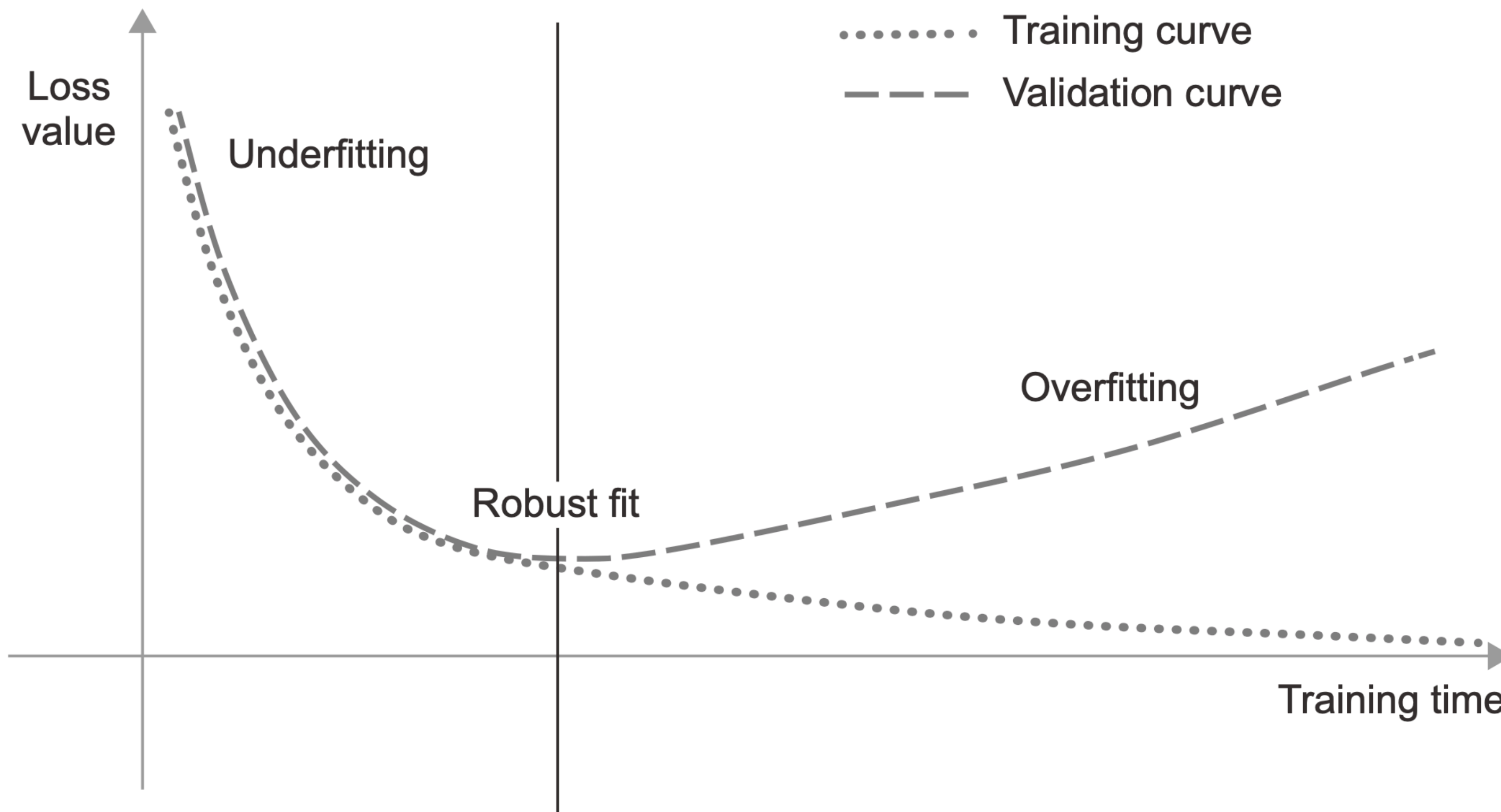


Figure 5.1 Canonical overfitting behavior

Robust fit vs. overfitting

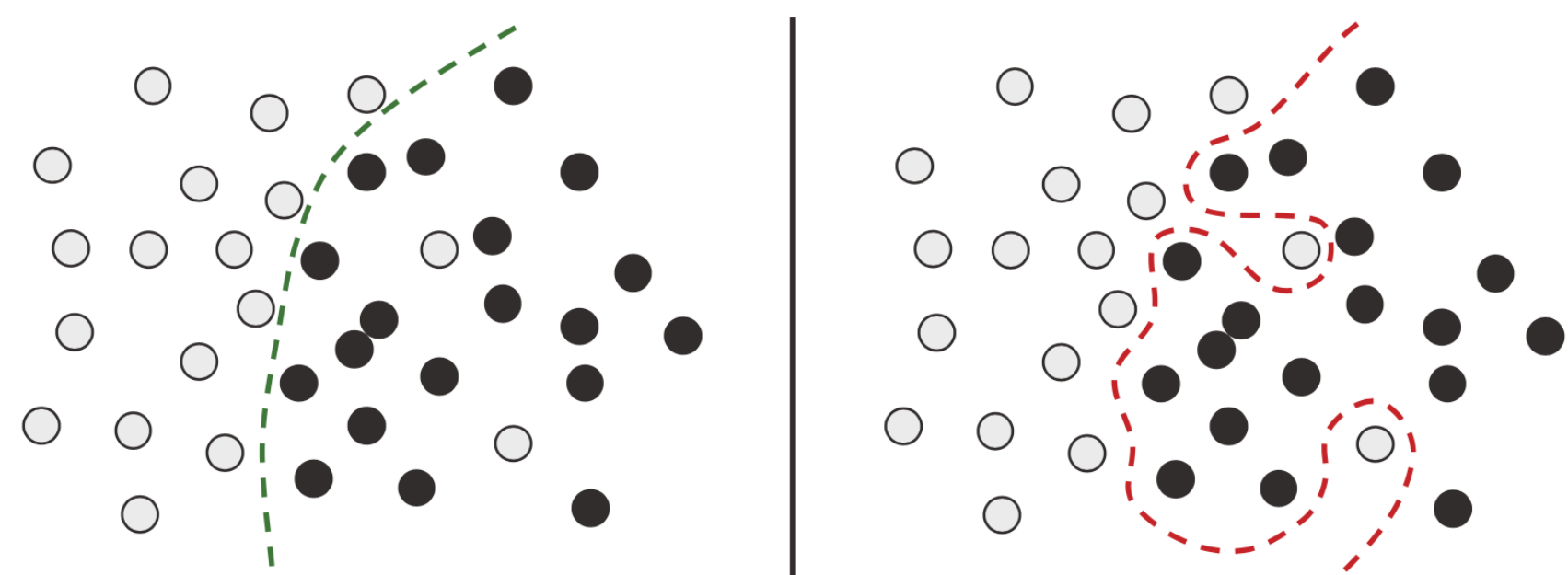


Figure 5.4 Dealing with outliers: robust fit vs. overfitting

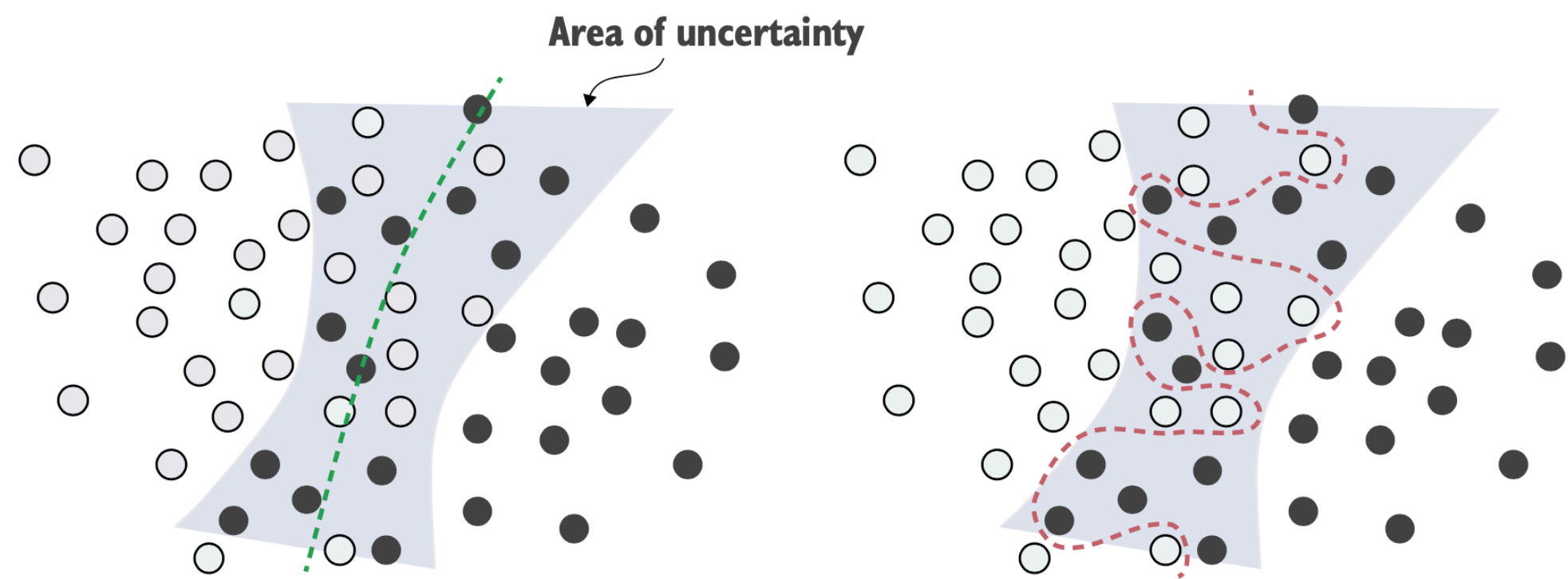


Figure 5.5 Robust fit vs. overfitting giving an ambiguous area of the feature space

Underfitting: training performance is bad

- This is the first thing we should worry about.

Overfitting: training performance is good, but test performance is bad

- This is the next thing we should worry about.

To prevent underfitting: change our model or make it general (to include the correct function in our solution set). Then train our NN well to find that correct function.

To prevent overfitting: after we have found a NN with good training performance, simplify the NN model (e.g., **reduce NN size**, or use **regularization techniques**). Or **get more data**.

Training data are very important

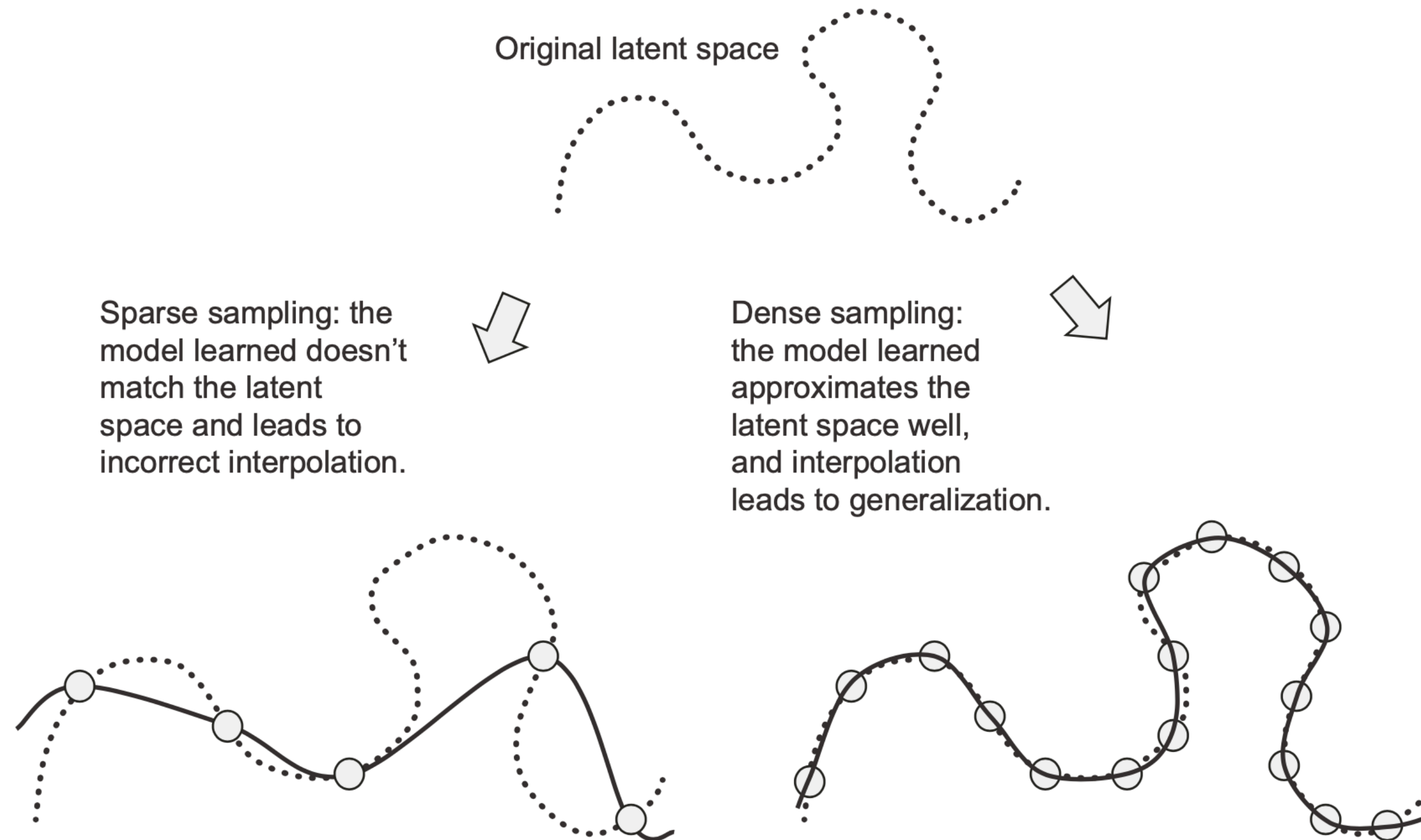


Figure 5.11 A dense sampling of the input space is necessary in order to learn a model capable of accurate generalization.

Quiz questions:

1. What is under-fitting?
2. What is over-fitting?
3. What is the importance of a good and large dataset?

Roadmap of this lecture:

1. Under-fitting and over-fitting.

2. Regularizations: L1, L2, dropout.



Regularization techniques

- **Weight regularization**: add a function of weights to the loss function, to prevent the weights from becoming too large.

L2 regularization new loss function = old loss function + $\lambda \sum_i w_i^2$

L1 regularization new loss function = old loss function + $\lambda \sum_i |w_i|$

A reason for weight regularization: large weight can make the model more sensitive to noise/variance in data.

L2 regularization: it tends to make all weights small.

L1 regularization: it tends to make weights sparser (namely, more 0s).

Listing 5.13 Adding L2 weight regularization to the model

```
from tensorflow.keras import regularizers
model = keras.Sequential([
    layers.Dense(16,
                  kernel_regularizer=regularizers.l2(0.002),
                  activation="relu"),
    layers.Dense(16,
                  kernel_regularizer=regularizers.l2(0.002),
                  activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
```

In the preceding listing, `l2(0.002)` means every coefficient in the weight matrix of the layer will add $0.002 * \text{weight_coefficient_value} ** 2$ to the total loss of the model. Note that because this penalty is *only added at training time*, the loss for this model will be much higher at training than at test time.

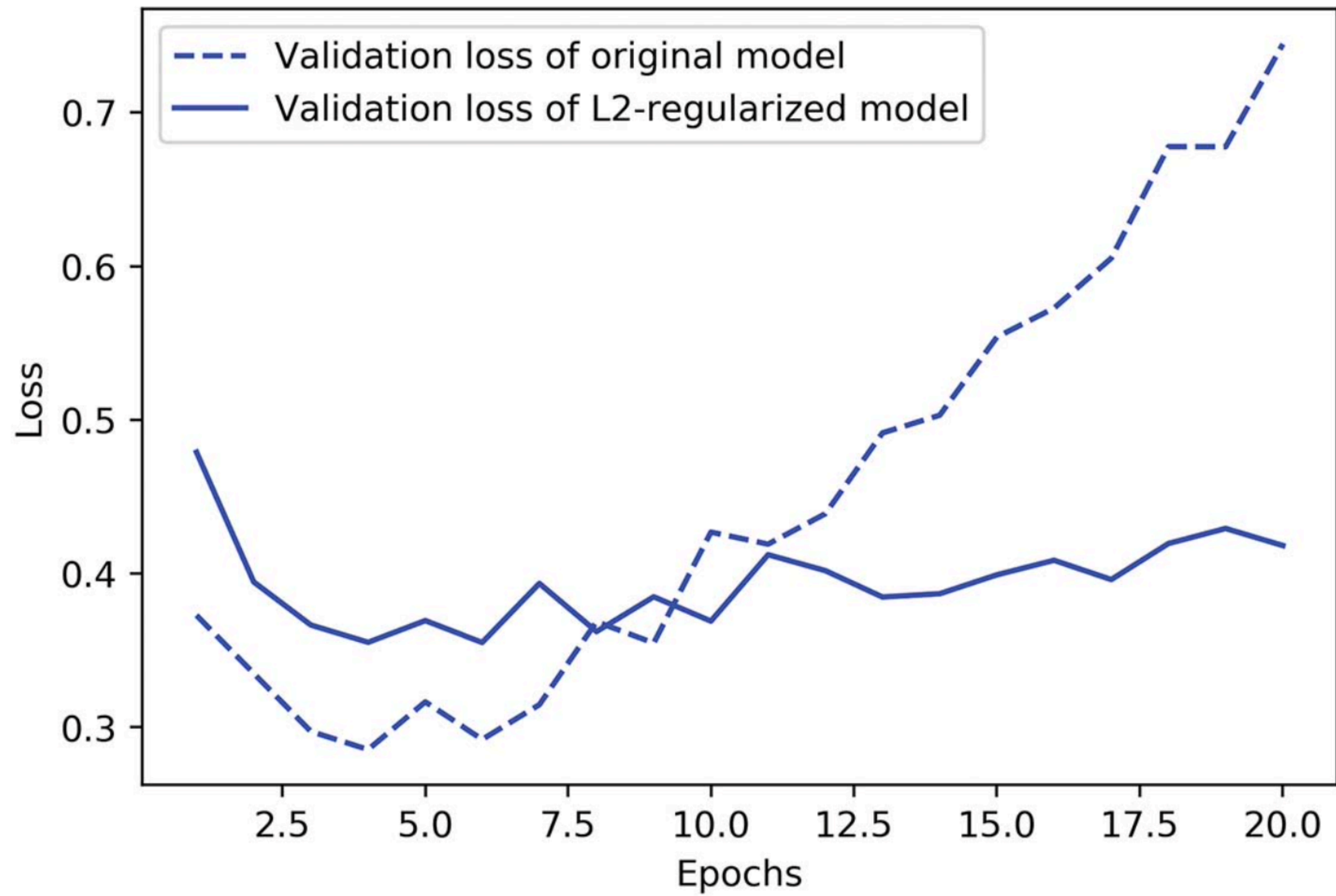


Figure 5.19 Effect of L2 weight regularization on validation loss

As an alternative to L2 regularization, you can use one of the following Keras weight regularizers.

Listing 5.14 Different weight regularizers available in Keras

```
from tensorflow.keras import regularizers
regularizers.l1(0.001)
regularizers.l1_l2(l1=0.001, l2=0.001)
```

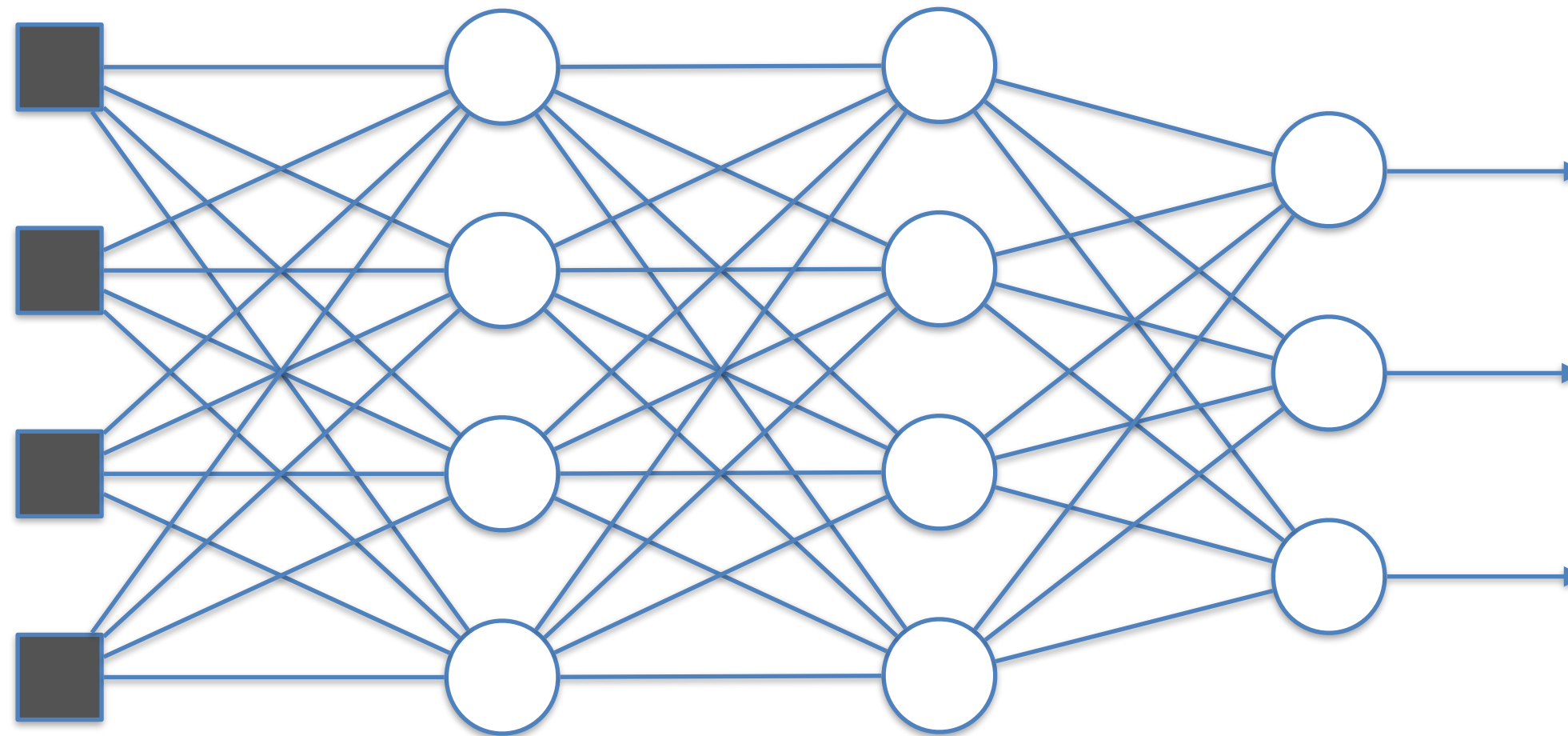
L1 regularization

Simultaneous L1 and L2 regularization

Note that weight regularization is more typically used for smaller deep learning models. Large deep learning models tend to be so overparameterized that imposing constraints on weight values hasn't much impact on model capacity and generalization. In these cases, a different regularization technique is preferred: dropout.

Dropout

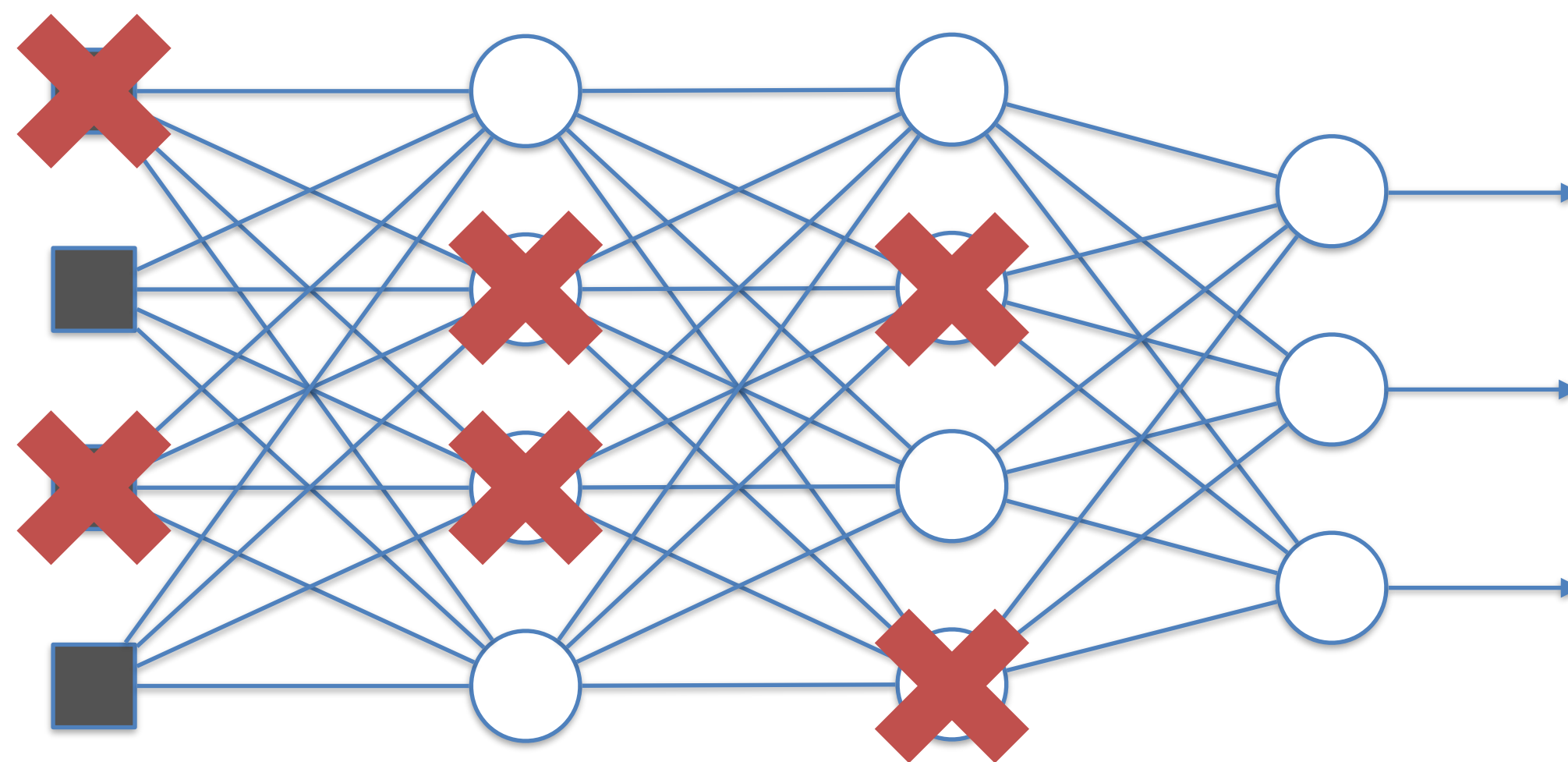
Training:

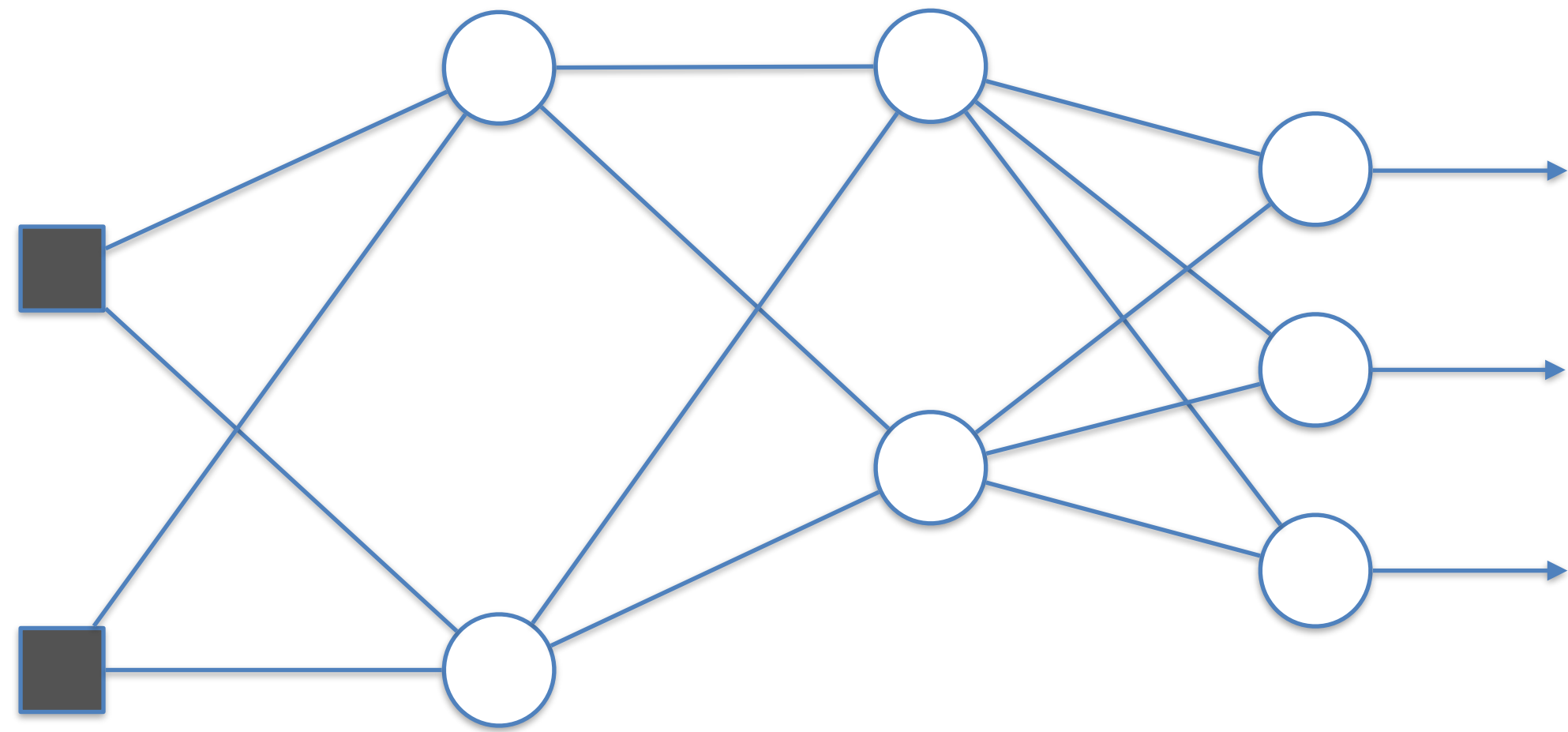


At every training step, each neuron has a probability p to dropout.

When a neuron drops out, all its edges are removed.

The dropouts at different training steps are independent.





Add Dropout

In Keras, you can introduce dropout in a model via the Dropout layer, which is applied to the output of the layer right before it. Let's add two Dropout layers in the IMDB model to see how well they do at reducing overfitting.

Listing 5.15 Adding dropout to the IMDB model

```
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history_dropout = model.fit(
    train_data, train_labels,
    epochs=20, batch_size=512, validation_split=0.4)
```

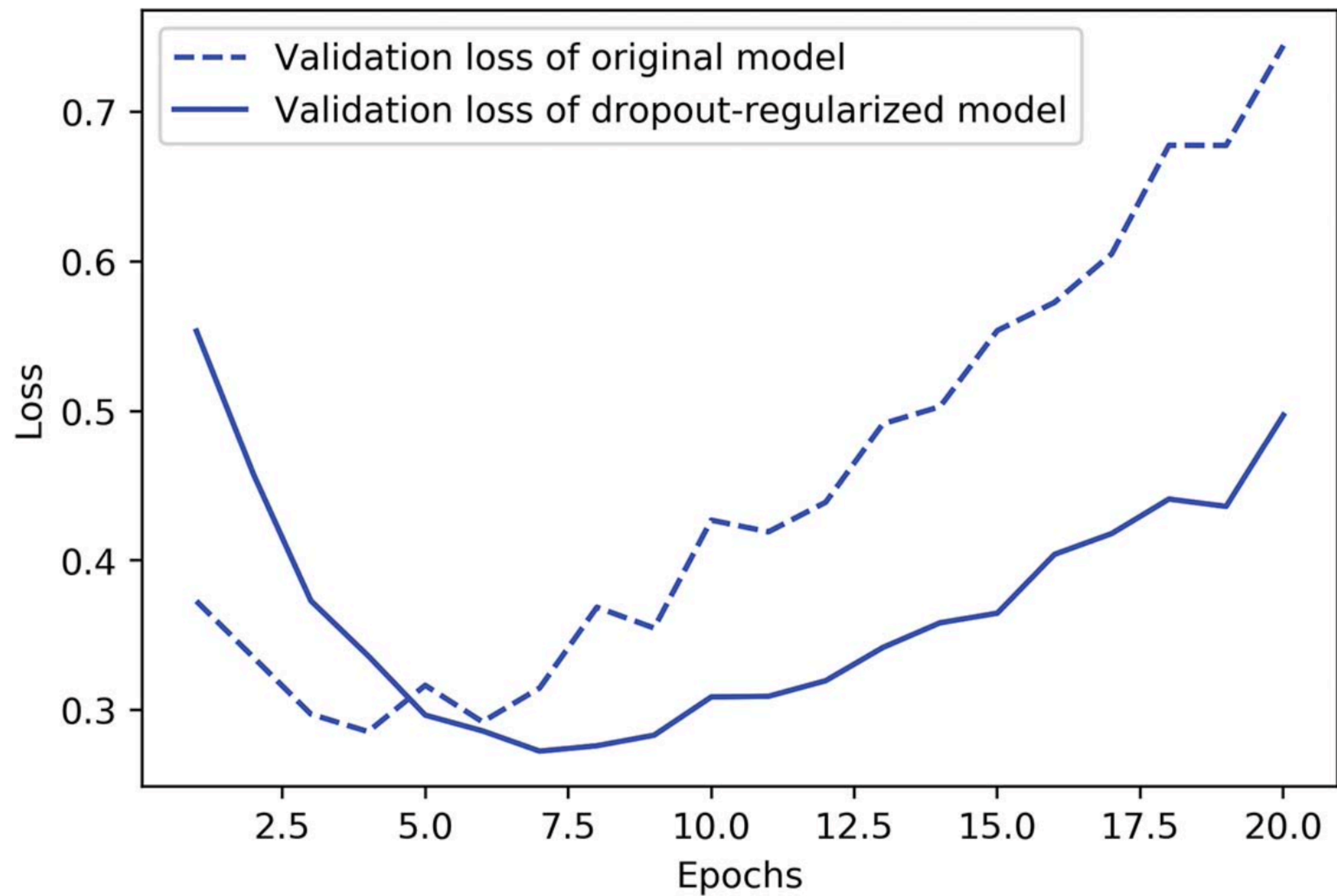


Figure 5.21 Effect of dropout on validation loss

Quiz questions:

1. What is the difference between L_1/L_2 regularization and dropout?
2. When should we use regularization?